

CS516/CSL516: Parallelization of Programs

Dependency Analysis

Vishwesh Jatala

Assistant Professor

Department of CSE

Indian Institute of Technology Bhilai

vishwesh@iitbhilai.ac.in



Recap

- Dependencies among the statements (simple)
 - ▣ RAW
 - ▣ WAW
 - ▣ WAR

Loop Dependencies

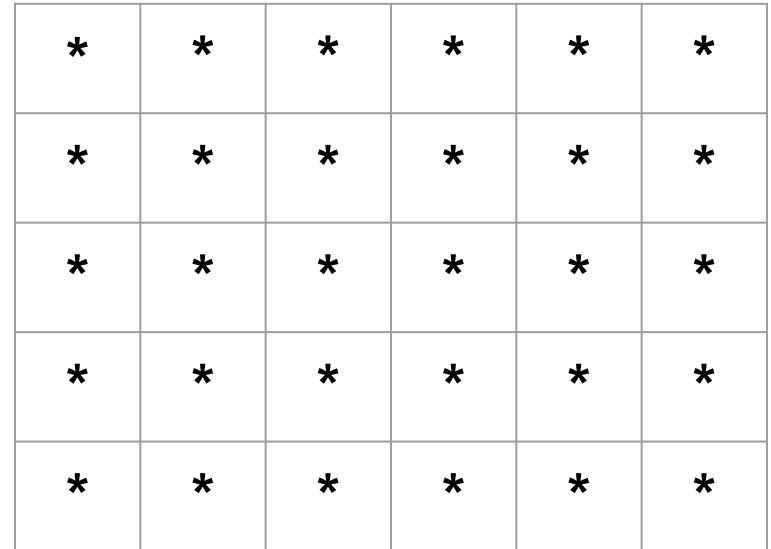
```
    for (i = 1; i<=N; i++)  
        for (j = 1; j<=M; j++)  
S1:      A[i+1][j] = A[i][j] + 10
```

We need a method to formalize the loop dependencies!

Iterations

```
for (i = 1; i<=6; i++)  
    for (j = 1; j<=5; j++)  
S1:    /*statement*/
```

j



*	*	*	*	*	*
*	*	*	*	*	*
*	*	*	*	*	*
*	*	*	*	*	*
*	*	*	*	*	*
*	*	*	*	*	*

i

- *The set of all possible iterations*
 - $(1,1) (1,2) \dots (1,5)(2,1)(2,2),\dots(5,5)(6,1)(6,2)\dots(6,5)$

Iteration Vector

```
for (i = 1; i<=6; i++)  
    for (j = 1; j<=5; j++)  
S1:    /*statement*/
```

j

*	*	*	*	*	*
*	*	*	*	*	*
*	*	*	*	*	*
*	*	*	*	*	*
*	*	*	*	*	*

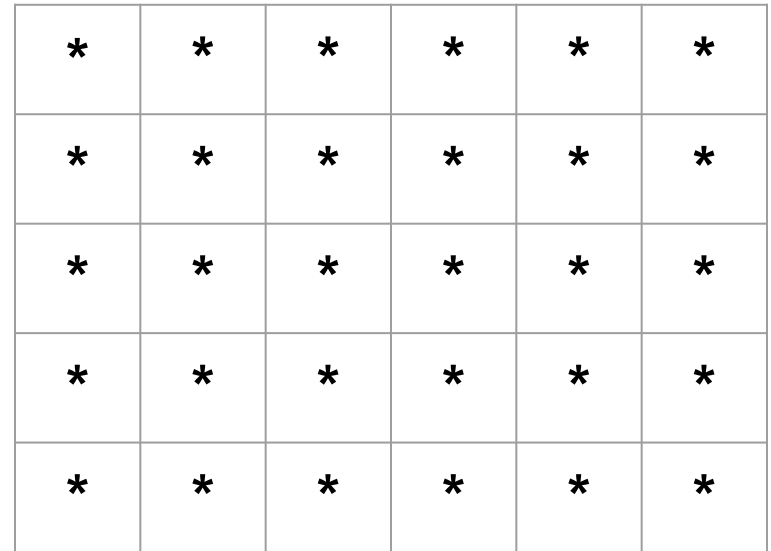
i

- The iteration vector S1[(3, 4)] denotes
 - the instance of S1 executed during the 3rd iteration of the i-th loop and the 4th iteration of the j loop

Iteration Space

```
for (i = 1; i<=6; i++)  
    for (j = 1; j<=5; j++)  
S1:    /*statement*/
```

j



*	*	*	*	*	*
*	*	*	*	*	*
*	*	*	*	*	*
*	*	*	*	*	*
*	*	*	*	*	*
*	*	*	*	*	*

i

- Iteration Space: The set of all possible iteration vectors for a statement
- Iteration Space of S1 is
 $\{(1,1) (1,2) \dots (1,5)(2,1)(2,2),\dots (5,5)(6,1)(6,2)\dots (6,5)\}$

Ordering of Iteration Vectors

- Useful to define an ordering for iteration vectors
- Define an intuitive, lexicographic order
- Iteration vector i precedes iteration vector j ($i < j$), if iteration i appears before iteration j in the lexicographic order

Iteration Vector Helps to Formulate Dependencies!

Loop Dependence Types

- If in a loop statement S2 depends on S1, then there are two possible ways of this dependence occurring:
 - S1 and S2 execute on different iterations
 - loop-carried dependence.
 - S1 and S2 execute on the same iteration
 - loop-independent dependence.

Loop Dependence Types

```
for (j = 1; j < 100; j++) {  
  S1:      a[j] =  
  S2:      = a[j-1]  
}
```

Loop Carried!

```
for (j = 1; j < 100; j++) {  
  S1:      a[j] =  
  S2:      = a[j]  
}
```

Loop Independent

Loop Dependence Types

```
for (j = 1; j < 100; j++) {  
  S1:      a[j] =  
  S2:      = a[j-1]  
}
```

Can we parallelize this loop?

```
for (j = 1; j < 100; j++) {  
  S1:      a[j] =  
  S2:      = a[j]  
}
```

If we execute the statements within each loop sequentially, then we parallelize this loop?

Loop Dependence Types

```
for (i = 0; i < 100; i++) {  
    for (j = 0; j < 100; j++) {  
        S1:      a[i+1][j] =  
        S2:      = a[i][j]  
    }  
}
```

Can we parallelize the inner loop?

Can we parallelize the outer loop?

We need a method to identify which loops are parallel

What we need?

- For a given loop nest (eg. i, j, k)
 - ❑ Need to identify if the dependence exist in each loop nest
 - ❑ Type of dependency
 - ❑ How to parallelize the loop nest?

Distance Vector

- Consider a dependence in a loop nest of n loops
 - Statement $S1$ on iteration i is the source of the dependence
 - Statement $S2$ on iteration j is the sink of the dependence
- The distance vector is a vector of length n $d(i,j)$ such that: $d(i,j)_k = j_k - i_k$

Example:

```
    for (i = 1; i<=N; i++)  
        for (j = 1; j<=M; j++)  
            for (k = 1; k<=R; k++)  
S1:          A[i+1][j][k-1] = A[i][j][k] + 10
```

- S1 has dependence on itself.
- Distance Vector = (1, 0, -1)

Direction Vector

- Suppose that there is a dependence from statement **S1** on iteration i of a loop nest of n loops and statement **S2** on iteration j , then the dependence direction vector is **$D(i,j)$** is defined as a vector of length n such that

$$D(i,j)_k = \begin{array}{ll} "<" & \text{if } d(i,j)_k > 0 \\ "=" & \text{if } d(i,j)_k = 0 \\ ">" & \text{if } d(i,j)_k < 0 \end{array}$$

Example:

```
    for (i = 1; i<=N; i++)  
        for (j = 1; j<=M; j++)  
            for (k = 1; k<=R; k++)  
S1:          A[i+1][j][k-1] = A[i][j][k] + 10
```

- S1 has dependence on itself.
- Distance Vector = (1, 0, -1)
- Direction Vector = (<, =, >)

Loop Carried Dependence

- Statement S2 has a loop-carried dependence on statement S1 if and only if S1 references location M on iteration i, S2 references M on iteration j and $d(i,j) > 0$ (that is, $D(i,j)$ contains a “<” as leftmost non “=” component).

Loop Carried Dependence

```
    for (i = 1; i<=N; i++)  
        for (j = 1; j<=M; j++)  
            for (k = 1; k<=R; k++)  
S1:                A[i+1][j][k-1] = B[i][j][k] + 10  
S2:                C[i][j][k] = A[i][j][k] + 2
```

Loop Carried Dependence

```
    for (i = 1; i<=N; i++)  
        for (j = 1; j<=M; j++)  
            for (k = 1; k<=R; k++)  
S1:                A[i+1][j][k-1] = B[i][j][k] + 10  
S2:                C[i][j][k] = A[i][j][k] + 2
```

- S1 and S2 are dependent.
- Distance Vector = (1, 0, -1)
- Direction Vector = (<, =, >) => Loop Carried

Loop Carried Dependence

- Level of a loop-carried dependence is the index of the leftmost non-“=” of $D(i,j)$ for the dependence.

```
for (i = 1; i<=N; i++)  
    for (j = 1; j<=M; j++)  
        for (k = 1; k<=R; k++)  
S1:      A[i][j][k+1] = A[i][j][k] + 10
```

- Direction vector for S1 is (=, =, <)
- Level of the dependence is 3
- A level-k dependence between S1 and S2 is denoted by $S1 \delta_k S2$

Loop Independent Dependence

- Statement S2 has a loop-independent dependence on statement S1 if and only if there exist two iteration vectors i and j such that:
 - 1) Statement S1 refers to memory location M on iteration i , S2 refers to M on iteration j , and $i = j$.
 - 2) There is a control flow path from S1 to S2 within the iteration.

```
for (j = 0; j < 100; j++) {  
  S1:      a[j] =  
  S2:      = a[j]  
}
```

Loop Independent Dependence

```
for (j = 0; j < 9; j++) {  
  S1:      a[j] =  
  S2:      = a[10-j]  
}
```

Does this have loop Independent dependence?

Exercise-1

```
DO I = 1, 100
  DO J = 1, 100
S1:    A(I+1,J) = A(I, J) + B(J)
  ENDDO
ENDDO
```

Compute the direction
vector

Exercise-2

```
DO I = 1, N
  DO J = 1, M
    DO K = 1, L
      A(I+1,J+1,K) = A(I,J,K) + A(I,J+1,K+1)
    ENDDO
  ENDDO
ENDDO
```

**Compute the direction
vectors**

Exercise-3

```
DO I = 1, 100
  DO J = 1, 100
S1:    A(I+1) = A(I) + B(J)
  ENDDO
ENDDO
```

Compute the direction
vectors

- The direction vectors for the dependencies is ($<$, $=$) ($<$, $>$) ($<$, $>$)
- * denotes union of all 3 directions
- The direction vector for the dependence is ($<$, *)
- If a loop index does not appear, its distance is unconstrained and its direction is “*”

References

- Slides of the book: Randy Allen, Ken Kennedy, Optimizing Compilers for Modern Architectures: A Dependence-based Approach, Morgan Kaufmann, 2001

Questions?

Thank you!